

Reliability assessment of AMD MicroBlaze-V TMR architecture using fault injection and proton irradiation

Jorge Cano-Páez^{*}, Luis Entrena, Almudena Lindoso

Dept. of Electronic Technology, Universidad Carlos III de Madrid, Leganés, Spain

ARTICLE INFO

Keywords:

Fault tolerance
FPGA
Microblaze-V
Proton
Radiation
RISC-V
Soft core

ABSTRACT

This work analyses the robustness of the recent AMD RISC-V processor: MicroBlaze-V. We study the TMR fault tolerant version of the MicroBlaze-V architecture, implemented in an SRAM-based FPGA. The architecture has been evaluated with fault injection and irradiation campaigns using 15 and 230 MeV proton beams. Experimental results show that AMD TMR MicroBlaze-V architecture offers a fast and flexible design process with good error mitigation of faults in the configuration memory. The results obtained show a good correlation between fault injection and proton irradiation campaigns, which may be used for error-rate estimations and precise error mitigation strategies. Overall, AMD TMR MicroBlaze-V has an error detection rate of around 99.84%. Approximately 1% of failures are not warned beforehand, although experimental data show that these errors are related to the memory, which is triplicated. It is calculated that, under the evaluated conditions, the mean time for a system failure is twice the mean time for a non-fatal error to happen. The cross-section is estimated to be 47 and 65 times better for failures with a previous warning compared to failures without warning in experiments with protons of 15 MeV and 230 MeV respectively.

1. Introduction

The New Space Era has fundamentally reshaped the development of high-reliability space systems, promoting a shift from traditionally costly and specialised components towards more accessible, flexible, and scalable technological solutions. Reliability also increasingly concerns other industries, such as automotive, where stringent safety and performance requirements of recent applications like autonomous driving [1], demand innovative and reliable design solutions. As technology advances, there is a growing emphasis on systems based on Commercial Off-The-Shelf (COTS) components and open-source architectures like those based on RISC-V, which aim to enhance flexibility while maintaining robust functionality [2].

COTS components offer a compelling alternative to custom-built hardware by combining cost-effective and readily available solutions with great performance. When combined with open and modular instruction set architectures (ISAs) such as RISC-V, they enable a configurable design suitable for demanding applications while reducing Intellectual Property (IP) licensing costs. The open nature and broad set of extensions of RISC-V provide flexibility to develop optimised solutions for diverse applications. Moreover, Field-Programmable Gate

Arrays (FPGAs) vendors are including RISC-V, providing soft-core implementations that can be deployed on their devices. Likewise, the growing interest in RISC-V by the industry resulted in the development of more mature software tools to unify the design, debug and validation of RISC-V based systems.

The space industry is currently focused on adopting RISC-V because of the benefits it provides in a sector characterized by low volume production. When implemented as soft-cores on reconfigurable hardware, RISC-V processors offer a practical alternative to the long and costly development cycles of application-specific hardware. Implementing RISC-V processors as soft-cores on reconfigurable platforms provides a practical approach to the design of newer systems, as these designs can be rapidly updated or migrated to new FPGA technologies. This enables a faster adoption of cutting-edge semiconductor advancements without requiring complete hardware redesigns. However, despite the advantages of COTS and RISC-V, ensuring the robustness of these designs remains a critical concern, particularly in radiation-intensive environments such as space. The configuration memory of SRAM-based FPGAs is susceptible to radiation-induced upsets that can produce critical errors because they change the original circuit configuration. Consequently, the assessment of the reliability of SRAM-based

^{*} Corresponding author.

E-mail addresses: jocanop@ing.uc3m.es (J. Cano-Páez), entren@ing.uc3m.es (L. Entrena), alindoso@ing.uc3m.es (A. Lindoso).

<https://doi.org/10.1016/j.micpro.2026.105301>

Received 9 January 2026; Received in revised form 15 May 2026; Accepted 10 June 2026

Available online 12 June 2026

0141-9331/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

FPGA designs with respect to configuration memory faults is essential.

In addition to the recent and standard RISC-V soft-core processor, the MicroBlaze-V, AMD provides a fault-tolerant variant hardened with Triple Modular Redundancy (TMR). This work studies the robustness of this fault tolerant architecture, implemented in a CMOD-A7 board with an Artix-7 35T FPGA (XC7A35T-1CPG236C) [3] through both fault-injection experiments and proton irradiation campaigns using 15 and 230 MeV protons. The resulting data allow for a detailed error classification and comprehensive reliability assessment of the evaluated system.

The remainder of this paper is structured as follows: Section 2 summarizes the related work; Section 3 describes the MicroBlaze-V TMR architecture, the fault-tolerance mechanisms provided by AMD, and the experimental design used in the experiments; Section 4 presents and discusses the experimental results; and Section 5 summarizes the conclusions of this study.

2. Related work

Reliability evaluation using fault injection is well known and has been used for a long time because of its flexibility and cost effectiveness. Consequently, many proposals have been made to improve fault injection mechanisms and platforms. There are two main fault injection methods. As a first approach, simulation-based fault injection [4] can be made due to its convenience and low cost, although it requires a large amount of time compared to other methods. In emulation-based fault injection, like in [5–8], a programmable circuit is used to accelerate the evaluation. Mechanisms like partial reconfiguration and circuit instrumentation are used to perform fault injection in implemented designs. Results show a good error coverage and testing, being able to perform fast injections and good estimations of the error rates of integrated circuits [9]. These methods are powerful, but developing the fault injection platforms may be complex. For high-reliability applications working in harsh environments like space, irradiation campaigns are needed to assess the robustness of the system. Thus, more simple mechanisms of performing fault injection are desired, as fault injection may be used as a first approach to the irradiation campaign. Modern FPGAs allow to perform fault injection in specific areas of the design using their internal configuration capabilities. AMD developed a convenience way of achieving this with the use of the Soft-Error Mitigation (SEM) IP [10]. This component allows to mitigate errors in the configuration memory of the FPGA, and it is also feasible to use it as a fault injection interface to the internal configuration mechanism.

Regarding reliability testing of RISC-V architectures, many recent works study RISC-V because it is an emerging architecture that is becoming attractive in diverse applications [11]. In [12], fine-grained fault injection is used to evaluate the reliability of a RISC-V soft-core, where different modules are analysed to obtain a comprehensive fault evaluation. Although not all studies are related to radiation-induced errors, results from [13] can be extrapolated from protection and countermeasure against deliberate attacks, to mitigation methods for high-reliability applications. Studies of the reliability of fault tolerant RISC-V architectures are presented in [14,15], where functional verification and fault injection are used. In [16], fault tolerant techniques for the RISC-V NOEL-V processor in a COTS device are evaluated with fault injection and proton irradiation. Moreover, the latter study is related to the effect of cache size, which proves that a bigger L1 cache with protection reduces the susceptibility of radiation-induced faults compared to smaller caches without protection.

3. MicroBlaze-V TMR architecture

The RISC-V architecture available in AMD environment is a soft-core optimised for their SRAM-based FPGAs called MicroBlaze-V IP [17]. It follows the same implementation scheme as previous MicroBlaze IP soft-core [18], and the fixed feature set of the processor uses the 32-bit

integer ISA in a pipeline architecture. It supports RV32I along with the most relevant extensions, including MAFC. The IP can be optimised for area, throughput, performance or frequency, which results in different number of pipeline stages, namely 3, 4, 5, and 8 stages respectively. A local memory or a shared one can be implemented with a local memory bus or an AXI interface respectively. MicroBlaze-V offers a program trace information for debugging purposes, with a configurable trace buffer. The associated development environment, Vitis IDE, has a debug environment that makes the development process faster. Also, the core is programmed via the Vitis IDE. Bootloaders and system configuration are made this way, allowing to either store program data into DDR memory or the FPGA BRAM. To summarise, Xilinx-AMD offers an all-in-one environment for fast development of RISC-V architectures using MicroBlaze-V IP.

Using the TMR Manager IP [19] in combination with MicroBlaze-V, the architecture can be made fault-tolerant. It implements a highly configurable TMR version of the architecture. There are three main possible architectures using TMR Manager IP:

- **Fault Tolerant Subsystem – Local Memory:** Three subsystems are in TMR, which contain the MicroBlaze-V core, a local BRAM, an I/O module and majority voters that take as inputs the outputs of all cores. The outputs of the I/O module are connected to the inputs of a non-redundant majority voter. This architecture provides high error protection as every module is redundant, including the memory itself.
- **Fault Tolerant Subsystem – ECC Memory:** It is the same architecture as before, but the memory is outside the TMR domain and protected with an Error Correcting Code (ECC). The memory is accessed through majority voters. In this configuration, a memory scrubber is recommended, as it is only protected with ECC and faults can accumulate. This architecture reduces resource usage, but it is less robust than the previous architecture.
- **Fail-Safe Subsystem:** This architecture goes a step beyond the Fault Tolerant Subsystem as each instance contains the MicroBlaze-V core, a local BRAM, an I/O module, majority voters, a TMR Manager module and comparators that are connected to the TMR Manager. The TMR Manager is in charge of error detection by observing the program trace of the cores and the outputs of comparators and voters. The local BRAM can also be configured with ECC in TMR, offering a higher fault tolerance.

Next, we will describe the architecture design selected as case study for this work together with the architectural choices made. In Fig. 1, the TMR MicroBlaze-V architecture designed (blue section) is shown. The architecture we selected when using TMR Manager IP is the Fail-Safe Subsystem, as it is the most robust. Three separate instances are created using the TMR Manager IP, which are represented as “MBV TMR instance” in the figure. Each instance contains a MicroBlaze-V (MBV) core, a local memory, a TMR Manager, a trace comparator and majority voters to access the local memory. The MicroBlaze-V core used is the one optimised for performance, which has a pipeline of five stages: fetch, decode, execute, memory access and writeback. The soft-core implements the RISC-V RV32I Base Instruction Set Architecture (ISA) with the extension for integer multiplication and division (RV32IM ISA). The local memory is implemented in BRAM and accessed through the local memory bus. Both instructions and data are stored in the local memory, represented as the blue and red lines respectively. As an error mitigation mechanism for the memory, a Single-Error-Correction Double-Error-Detection (SECEDED) code is applied. This code is generated when configuring the local memory. The local memory of each instance is accessed with majority voters that take the input from all instances, but its output is only directed to the MBV core inside the instance.

The TMR Manager monitors the system using the output of a comparator that takes as input the program trace, represented by green lines in the figure, and the uncorrectable memory error signal given by

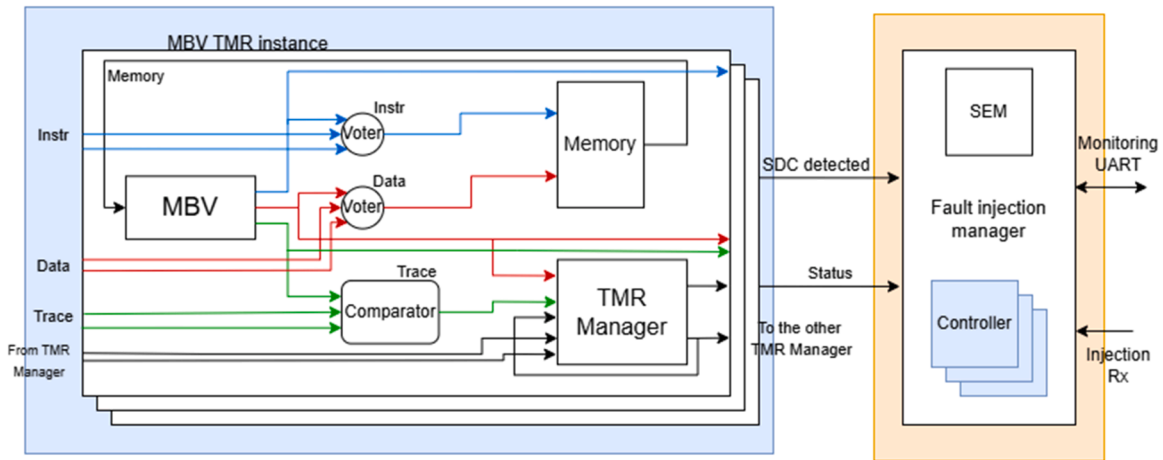


Fig. 1. MicroBlaze-V TMR Fail-Safe architecture evaluated.

the SECDED decoder. Because the program trace of each core is connected to all TMR Managers, it is possible to know which instance fails in case of an error. Furthermore, to obtain updated and correlated information, the TMR Manager outputs monitoring information to other TMR Managers. The status of the system is given by the “Status” signal. This architecture is designed in a block TMR scheme, but monitoring the program trace offers a more detailed way of detecting errors for fault tolerant capabilities. Lastly, although the SEM IP can be used with the TMR Manager for testing and fault tolerant purposes, we have only used it for fault injection. It is implemented in the fault injection manager module shown at the right part of Fig. 1 and will be explained later in Section 4.

3.1. System status

The TMR Manager IP has three modes of operation depending on which error happens and the previous mode:

- **Fault Tolerant mode.** No error has been detected and all three TMR MBV instances are working correctly.
- **Lockstep mode.** An error that can be recovered has been detected and two out of three TMR MBV instances are working correctly.
- **Fatal mode.** The TMR Manager has detected an unrecoverable error, and the system ceases operation.

The system starts in Fault Tolerant mode and monitors the program trace to check if an instance fails. When all TMR Managers identify the same faulty instance, the system changes to Lockstep mode. In Lockstep mode, the nominal operation of the entire architecture is ensured. In this mode, a recovery process is needed to restore the faulty instance and go back to Fault Tolerant mode. If another instance fails or an uncorrectable memory error happens while in Lockstep mode, the system goes into Fatal mode. The system can also change from Fault Tolerant mode to Fatal mode if an illegal combination of comparators is detected or an uncorrectable memory error happens. The Status signal given by the TMR Manager offers information of the system operation. When an error affects a core, the Status signal indicates which MicroBlaze-V instance fails. Moreover, this signal also tells if a majority voter fails, if there is a comparator mismatch or if there is an uncorrectable memory error. A distinction between non-critical errors and fatal errors is also made. The signal itself is not used by the TMR Manager but it can be used to gather information of the system. Software or hardware-based error recovery processes can be implemented. Moreover, a specialised recovery module could be implemented in the design to perform the recovery process triggered by the Status signal.

4. Experimental results

For clarity, this section is organised as follows. First, an explanation of the design tested during experiments is given, followed by the error classification used to present the results. Then, the experimental data gathered during the fault injection and the 15 MeV and 230 MeV proton irradiation campaigns are presented. Finally, an analysis and discussion of the results is done.

4.1. Experimental design

For the experimental design, an additional module is added. In Fig. 1 that module is noted as “fault injection manager”, which performs fault injection and data gathering. This module uses the SEM IP to perform fault injections in the configuration memory of the FPGA. It is designed as a TMR fault tolerant module with majority voters and the SEM IP being the only non-redundant element. We decided to design the fault injection manager as fault tolerant, because during irradiation testing all parts of the design are exposed to the effects of radiation and can fail. This way, the same implementation can be tested with fault injection and irradiation even though the fault injection manager is not part of the architecture to be tested. The fault injection manager also takes as input the Status signal from each MBV TMR instance. To make fault injections and to gather data, the system is connected to a Raspberry PI via UART (monitoring UART and injection UART in Fig. 1). The benchmark selected for execution is the CoreMark program, which was modified so it could be stored in BRAM memory. It occupies 21.852 KB. No software hardening techniques are used. The CoreMark executes two sub-benchmarks sequentially: matrix operations and linked lists operations. After a sub-benchmark is done, the cores check a signature calculated throughout the execution with a pre-computed golden signature to detect SDCs. The system in Fig. 1 was implemented in a CMOD-A7 board with an Artix-7 35T (XC7A35T-1CPG236C) FPGA [3] working at 70 MHz clock frequency.

Resource utilisation is shown in Table 1. Each table cell shows the number of resources used by the module given in the first column and the percentage relative to available resources. The row MBV TMR represents the MBV TMR instances seen in Fig. 1. They use most of resources, and they alone make the MicroBlaze-V TMR architecture. The following row, labelled “Core”, shows the resource utilisation of one MicroBlaze-V core without local memory. Thus, the overhead of the TMR architecture is around 4.5 for LUTs and 4.3 for registers compared to the MicroBlaze-V core alone. The fault injection manager along with other logic components are used only in the experimental design. The row labelled “Glue logic” refers to the rest of resources used to interconnect the modules. The last row shows the total available resources of

Table 1
Resource utilisation.

	LUTs	Registers	LUT as logic	LUT as memory	BRAM tiles
MBV TMR	5624 27.04%	3684 8.86%	5351 25.73%	273 2.84%	30 60%
Core	1258	857	1170	88	0*
Fault injection manager	1254 6.03%	1305 3.14%	1009 4.85%	245 2.55%	1.5 3%
Glue logic	122 0.59%	163 0.39%	121 0.58%	1 0.01%	0 0%
Device	20,800 100%	41,600 100%	20,800 100%	9600 100%	50 100%

* Local memory not included as part of the core.

the device. The architecture with local memory demands a large amount of BRAM because memory needs to be triplicated. Nevertheless, it is observed that MicroBlaze-V instances do not use many resources, making the IP a great resource-friendly soft-core solution optimised for AMD SRAM-based FPGAs.

4.2. Error classification

In our experimental results we have considered the following classification of errors:

- **Lockstep.** This error happens when the system is in Fault Tolerant mode and an error in one of the instances is identified. The system changes to Lockstep mode.
- **Fatal.** This error happens when the system has detected an error that forces the system to enter Fatal mode, thus stopping the execution.
- **DBE.** Double bit error that happens because of an uncorrectable ECC error in BRAM and drives the system into Fatal mode. This error is voted and only occurs when at least two instances signal an uncorrectable ECC error in BRAM.
- **SDC.** Silent data corruption. An error occurs in data, which affects computations, but the system keeps its functionality. This type of error cannot be detected unless specific checking mechanisms are used, like time redundancy. SDCs are detected by comparing the results with precomputed golden values. The comparators are voted to ensure SDC error detection is robust.

We consider a system failure when Fatal, DBE, or SDC errors occur.

4.3. Experimental data

All experiments were conducted without configuration memory scrubbing. For our study, this methodology is advantageous as analysing system reliability under cumulative fault conditions provides valuable insight for designing effective error recovery techniques. For example, the analysis can be used to determine the optimal configuration memory scrubbing frequency required to maintain correct system operation without resorting to excessive scrubbing, thereby reducing power consumption. Besides, performing a reliability analysis with a scrubber might overestimate the robustness of the tested architecture, as faults could be corrected by the scrubber before the architecture can fully exercise its own error mitigation mechanisms. In addition, although a scrubber can correct errors in the configuration memory, it is not sufficient for complete error recovery. For instance, in the tested TMR architecture, if an error affects the configuration memory of a soft-core processor, that core must be resynchronised with the remaining cores. A key objective of this work is therefore to analyse the reliability of the system between scrubbing cycles.

4.3.1. Fault injection campaign

We performed a fault injection campaign, using the essential bits of the design. Essential bits are the bits of the configuration memory that are directly used to implement an FPGA design. Not all configuration memory bits are necessarily used, as not all FPGA resources are used by a design. Therefore, performing fault injection in essential bits is preferred to injecting all the configuration memory bits because the probability of generating errors is higher, thus reducing the time needed. Nevertheless, injecting in all essential bits during different operational time windows requires an extensive testing time. As presented in [20], a statistical fault injection campaign can be done in order to reduce the number of injections while obtaining meaningful information of statistical relevance. The number of fault injections needed can be estimated following (1):

$$n = \frac{N}{1 + \frac{e^2(N-1)}{t^2p(1-p)}} \quad (1)$$

with N as the number of essential bits, e the error margin, t the critical value used for the confidence interval, p a statistical parameter that corresponds to an estimate of the percentage of errors resulting in a failure, and n the number of statistical injections needed. The number of essential bits in the design (N) is 1423,619. We chose an error margin (e) of 1%, a confidence level of 99.9% which corresponds to $t = 3.27$, and a p of 0.5 as expressed in [19]. The resulting statistical injections needed for the design is 26,239, but we decided to perform some more in order to obtain more data and better characterise the design. During the fault injection campaign, a total of 40,175 faults were injected which gives an error margin of 0.8%, resulting in 5463 errors (13.6% ratio) and 2744 runs. We consider a run from the beginning, where no injections are made, until a reset is done.

To perform the fault injection campaign with essential bits, a file containing a set of linear frame addresses of the essential bits was generated. It is important to mention that the essential bits of the fault injection manager were not injected, as it is only used for instrumentation and is not part of the architecture itself. This was achieved in Vivado, setting the design as partially reconfigurable. A floorplanning was made, setting the fault injection manager in a partially reconfigurable Pblock. We made a Python script that randomly selects an address and sends it to the SEM IP via UART.

Data regarding observed events are shown in Table 2. Each row shows the number of events observed for that type of error and its percentage with respect to the total number of events. The table is divided in two parts: the first one for events observed after a lockstep error occurred, and the second one for events without a previous lockstep error. The total count of events is also reported.

In around 99% of the failures observed, a lockstep error was detected before system failure. Most of the times a lockstep error was issued, it was due to a faulty MicroBlaze-V. Only in 0.15% of the cases, one TMR manager was detected as faulty. Of the three types of errors that can originate a failure, most are due to fatal errors detected by the TMR Manager (around 71%), followed by DBE (27.73%) and SDC (0.29%). Near 30% of errors are DBEs affecting the local memory. The memory is

Table 2
Errors observed with fault injection.

Error		Observed Events	Percentage
Previous Lockstep Error	Fatal	1950	71.06%
	DBE	761	27.73%
	SDC	8	0.29%
	Total	2719	99.09%
No previous Lockstep Error	Fatal	0	0%
	DBE	25	0.91%
	SDC	0	0%
	Total	25	0.91%
Total		2744	100%

protected with a SECEDED code, but for a DBE to appear, at least two of the local memories must be affected. This result shows that BRAMs are also susceptible to faults in the configuration memory, as there are configurable routing and logic needed to access them. We also observed SDCs, and although the incidence is low, with only 8 SDCs observed, they are critical. These errors pose a great threat to the system, because the system generates incorrect data while no problem is detected. Monitoring the trace is a reliable way of detecting errors, but there are occasions in which data corruption might not be detected and could affect system functionality.

Around 1% of observed events happened without issuing a previous lockstep error. The optimal process would be to first issue a lockstep error, which serves as a warning because the system has one failing instance and loses the error-correcting capability of the TMR architecture. With a warning, the system could start error recovery actions, but results show that about 1% of the times, this was not the case. All errors without a previous lockstep error indication were due to DBE.

In comparison with the error rate reported in [16] for the NOEL-V soft-core with distributed TMR, these results show an SDC rate of 2×10^{-4} , which is approximately one order of magnitude lower. The functional error rate is 4.9×10^{-2} , which is about one order of magnitude higher than that reported in [16]. It should be noted, however, that the technology used in [16] is different (UltraScale+), and the benchmark also differs (matrix multiplication). Nevertheless, the MicroBlaze-V Fail-Safe architecture provides a warning feature that allows all SDCs and functional errors to be detected in advance.

4.3.2. 15 MeV proton irradiation campaign

A proton irradiation campaign was done at Centro Nacional de Aceleradores (CNA) in Seville, Spain. The proton energy was 15 MeV with an average flux of 6.43×10^7 p/cm²/s, which resulted in a fluence of 2.72×10^{11} p/cm². Results obtained in this campaign are presented in Table 3. This table follows the same structure as in fault injection case (Table 2). Of all failures, around 99% of them were warned beforehand by a lockstep error, which shows the high error mitigation that monitoring the program trace provides. This subset of errors is made of around 74% Fatal errors, followed by 26% of DBEs. Errors that were not preceded by a previous lockstep error made around 1% of the total, being two DBEs. A notable aspect of these results is that no SDC was found during the whole experiment. Therefore, the architecture showed a 100% error detection rate for the proton irradiation campaign in the exposure time of the test. SDCs could eventually occur with extended exposure time, but this result confirms that the SDC rate is low.

We calculated the 95% confidence interval of the cross-section. The rightmost column of Table 3 shows the upper limit of the confidence interval. From these data, two key results are extracted: the total cross-section of the design is 1.72×10^{-9} cm², for failures is 8.97×10^{-10} cm², and the cross-section of errors that were not warned beforehand is 2.66×10^{-11} cm². The cross-section of errors without a previous lockstep is 64.5 times lower than the total cross-section. Using SPENVIS tool, for an

11-year mission in a 1000 km orbit around the Earth, the integral proton fluence over 15 MeV is approximately 1.44×10^{11} p/cm², which is close to the fluence obtained in the irradiation campaign. An approximate result for the Mean Time To Failure (MTTF) can be calculated with the cross-sections obtained. Following the 95% confidence interval, the MTTF for a failure with a previous lockstep is approximately from 31 to 41 days. For a failure without a previous lockstep error is around from 2 to 85 years.

4.3.3. 230 MeV proton irradiation campaign

Another proton irradiation campaign was done at Paul Scherrer Institute (PSI) in Villigen, Switzerland. The proton energy was 230 MeV with an average flux of 9.97×10^7 p/cm²/s, which resulted in a fluence of 4.28×10^{11} p/cm². Results obtained in this campaign are presented in Table 4, which follows the same structure as Tables 2 and 3. Around 98% of the failures were warned beforehand, providing great error mitigation capabilities. Of the failures that had a previous lockstep, Fatal errors make the 67.57%, followed by DBE around 30% and around 1% of SDCs. As opposed to the irradiation campaign with 15 MeV protons, SDCs were found in this irradiation campaign. This proves that the fault injection campaign is representative, even with events that have a low probability of happening. Lastly, failures that were not preceded by a previous lockstep error are <2%, with only 4 DBEs.

The rightmost column of Table 4 shows the upper limit of the 95% confidence interval. The total cross-section of the design is 1.13×10^{-9} cm², and for failures is 5.92×10^{-10} cm². The cross-section of errors that did not have a previous lockstep error is 2.39×10^{-11} cm², which is 47 times lower than the total cross-section. Using SPENVIS tool, for an 11-year mission in a 1000 km orbit around the Earth, the integral proton fluence over 230 MeV is approximately 3.77×10^{10} p/cm², which is around an order of magnitude lower than the fluence obtained in the irradiation campaign with 15 MeV protons. An approximation can be done in order to obtain the MTTF, which results in a failure with a previous lockstep error around 3 to 9 years and virtually no failure without a previous lockstep error during the mission duration. If a scrubber for the configuration memory was used, there should not be any failure, as 3 years is a time window large enough to remove any faults in the configuration memory that were accumulated during flight.

4.4. Discussion

A key point of the three experimental campaigns is that data obtained during fault injection and both proton irradiation campaigns correlate very well in the percentage of events observed. In all experiments, around 99% of all failures were preceded by a previous lockstep error. Moreover, the three types of error that produce failures also follow the same percentages. With a previous lockstep error, Fatal correspond to around 70%, DBE to slightly <30% and SDC to 1%. Without a previous lockstep error, only DBEs were reported. To further analyse the correlation between fault injection and irradiation, we calculated the

Table 3
Errors observed in 15 MeV proton irradiation campaign.

Error	Observed Events	Percentage	Cross-section (cm ²)
Previous Lockstep Error	Fatal	157	73.71%
	DBE	54	25.35%
	SDC	0	0%
	Total	211	8.89 × 10 ⁻¹⁰
No previous Lockstep Error	Fatal	0	0%
	DBE	2	0.94%
	SDC	0	0%
	Total	2	0.94%
Total	213	100%	8.97 × 10 ⁻¹⁰

Table 4
Errors observed in 230 MeV proton irradiation campaign.

Error	Observed Events	Percentage	Cross-section (cm ²)
Previous Lockstep Error	Fatal	150	67.57%
	DBE	66	29.73%
	SDC	2	0.9%
	Total	218	98.2%
No previous Lockstep Error	Fatal	0	0%
	DBE	4	1.8%
	SDC	0	0%
	Total	4	1.8%
Total	222	100%	5.92 × 10 ⁻¹⁰

probability distributions of failures as a function of the injections (for the fault injection data) and exposure time (for irradiation data).

Firstly, by plotting the histogram of failures as a function of injections and exposure time, the lognormal-like behaviour of the data collected was observed, as shown in Fig. 2. This fact is supported by the effects of cumulative injections (or the equivalent irradiation exposure time) in the configuration memory of SRAM-based FPGAs when no error-accumulation mitigation technique is used. The accumulation of faults in the configuration memory can lead to errors due to their multiplicative effect by changing the implemented logic with a constant injection rate (or irradiation exposure time with a constant particle flux). To compute the distribution, we used maximum likelihood estimation (MLE), with which we obtained the mean and standard deviation parameters. Fig. 3 shows the probability density function (PDF) and cumulative density function (CDF) of experimental data, computed for failures as a function of injections and exposure time using the lognormal distribution. It is observed that experimental data for failures follow a specific distribution and that distributions correlate between them. This point proves that results from the fault injection campaign can be used to model the behaviour of the architecture under proton irradiation effects. This is a notable result of this research, as fault injection campaigns are more feasible than irradiation experiments due to difficulties in finding available beamtime and the cost of such experiments. Therefore, if fault injection can be used to model irradiation effects in a valid manner, more data and specific analyses can be done in order to better characterise systems. As such, we will continue this discussion with data from the fault injection campaign.

We will consider the lifetime as the time during which the system is available and producing valid results until a failure occurs. The lifetime of the system, dependent on injections in essential bits of the configuration memory is represented in Fig. 4. The graph shows the process explained in Section 3.1, in which the system starts in time t_0 and a different number of injections are needed to observe an error. The vertical lines represent error events; the numbers above the lines represent the mean value of injections needed for an error to occur; and the text below the lines represent the operation mode of the system at that moment. The modes are Fault Tolerant (FT), Lockstep (LS) and Fatal (FA). Starting in FT mode, for a lockstep error to happen, a mean value of 6.5 injections in essential bits is needed. When the system enters LS mode after a lockstep error, around 8 injections are needed on average to

produce a failure. Then, the system changes to FA mode and ceases operation. The percentage of errors that produce a failure is given to the right of the failure vertical line and are taken from Table 2. Although only around 1% of errors produce a failure without a previous lockstep error, the average number of injections needed for this to happen is 13. If a memory error correction mechanism such as configuration memory scrubbing is used, the periodicity of the scrubber should be, on average, higher than the average time for the occurrence of 13 faults in essential bits (the time is dependent on the real application and environment).

The injections needed for errors to occur are reported in Table 5. The table is divided into errors after a lockstep and those without a previous one. Data shown corresponds to the mean, the median and the number of errors observed. The percentage related to the number of errors is the same as in Table 2, which is relative to the total number of failures. Each row represents the injections needed for an error in the first column to happen. Injections are counted from the beginning of the run until an error happens. After the first lockstep error, the injection count is reset and started again until the next error is detected.

On average, 6.57 injections are needed for a lockstep error to happen, backed by the large number of events observed. While in lockstep mode, the number of injections needed for a fatal error to happen is slightly bigger than for lockstep error. After a warning is issued by the TMR Manager, 8.21 and 7.78 injections are needed for a fatal and DBE error to happen respectively. Therefore, the lifetime of the system is around the double of the time for a warning to appear. This is a relevant result to implement error correction and recovery techniques, as it gives a time window to operate before the system fails. For example, after a time t the system suffers an error that makes it enter Lockstep mode. Then, the system has around the same time t to perform error correction and recovery techniques before it suffers a failure. If the system is performing a critical task that cannot be interrupted even for error recovery purposes, this knowledge of the behaviour can be used to decide which strategy to follow in critical scenarios. If relevant statistical data of the error distribution is known, the correct performance of the application could be ensured. On the other hand, SDCs are a concern even though very few SDCs were observed. The mean value of injected faults after a lockstep error is 5.63, although only 0.29% of failures are SDCs. The rightmost half of the table shows the number of injections needed for a failure without a previous lockstep error. As shown in Section 4.3.1, only DBEs were observed without a previous lockstep

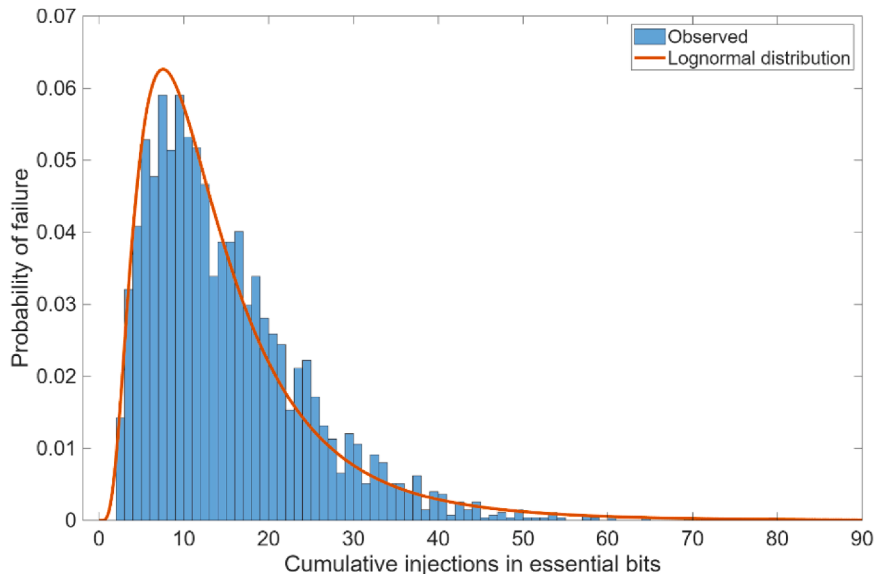


Fig. 2. Histogram of observed failures as a function of injections compared to a fitted lognormal probability density function (PDF).

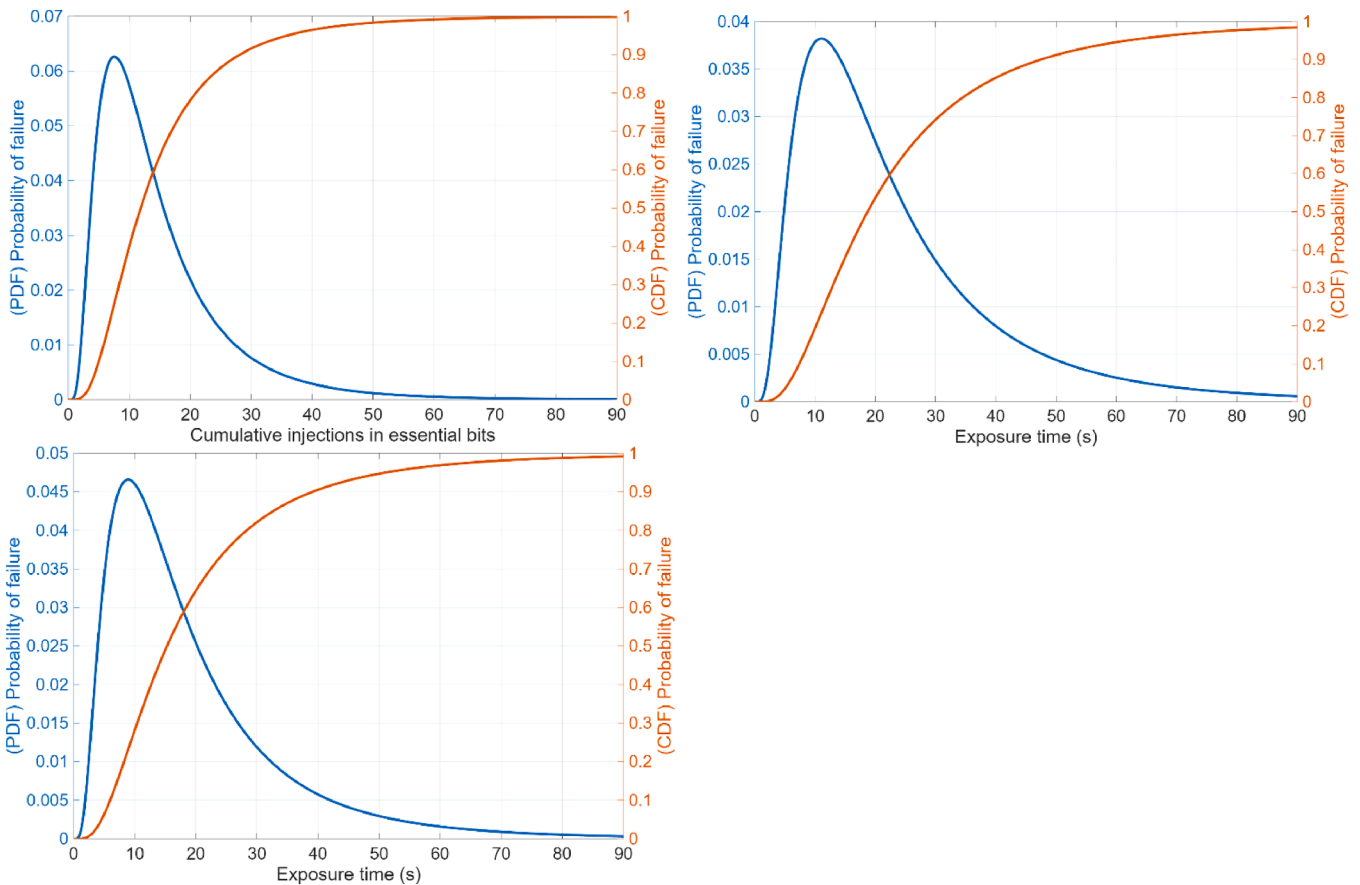


Fig. 3. Probability density function (PDF) (in blue) and cumulative distribution function (CDF) (in orange) of failures. From upper left to lower right PDF and CDF of failures in: a) fault injection campaign, b) 15 MeV proton irradiation campaign, c) 230 MeV proton irradiation campaign.

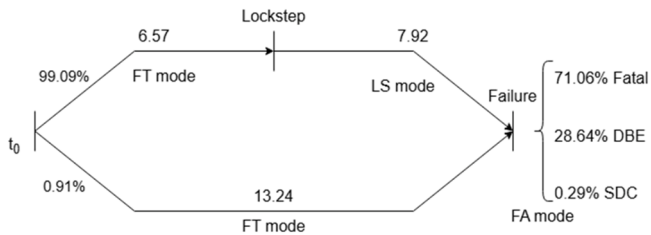


Fig. 4. Lifetime of the system when different types of errors occur.

Table 5

Injections needed for errors to occur.

	Lockstep			No lockstep		
	Mean	Median	#	Mean	Median	#
Lockstep	6.57	5	2719 99.09%	-	-	0 0%
Fatal	8.21	6	1950 71.06%	-	-	0 0%
DBE	7.78	5	761 27.73%	13.24	12	25 0.91%
SDC	5.63	3.5	8 0.29%	-	-	0 0%

error. Although $<1\%$ of all failures did not enter lockstep mode, the number of injections needed to happen is around double the time for a lockstep error to appear.

Another advantage of obtaining the distribution of errors that the design follows is that it allows to design more specific error mitigation techniques backed by statistics. A common procedure when using scrubbers for space applications is to obtain the error rate or MTTF and, as a rule of thumb, set the scrubbing frequency to 10x or 100x the expected error rate. This safety margin in frequency helps to reduce error effects at the expense of power consumption. Block TMR designs have the ability to mitigate errors caused by few SEUs as the non-faulty blocks can continue operation while one has been affected. This is a well-known characteristic of Block TMR, which is shown in Section 4.3. As such, setting a high scrubbing frequency increases the power consumption without providing a real value when dealing with failures, although it helps to reduce fault accumulation in the configuration memory.

In the distribution computed for failures in Fig. 3.a, the mean value of injections needed for a failure is around 11.87. If a scrubber is used with a scrubbing frequency 10 times this rate, the probability of failure for 1.187 injections is 0.03%. This result cannot be known without calculating the distribution. Secondly, depending on the criticality of the application, this result might be accepted or not. For a highly critical application, this probability could be a risk, for which a new scrubbing frequency can be selected thanks to the distribution. On the other hand, for a less critical application, the probability of failure could be acceptable at a 1%. In this case, the scrubbing frequency should be that

of the rate of around 2.481 injections, that is 4.78 times higher than for 50% probability of failure and 2.09 times slower than the frequency needed for a probability of failure of 0.03%. In this example, for a constant radiation flux, using the 1% probability of failure scrubbing frequency would mean a reduction of 50% in power consumption compared to the 10x scrubbing frequency. Thanks to the obtained error distribution, an optimal balance between mission reliability requirements and power consumption overhead can be found.

5. Conclusions

We have performed fault injection and proton irradiation campaigns with energies of 15 MeV and 230 MeV to evaluate the reliability of the TMR architecture of MicroBlaze-V over faults in the configuration memory. Experimental results show a great correlation between the fault injection and irradiation campaigns for the CoreMark benchmark. Moreover, the theoretical distribution of errors has been calculated to provide better error mitigation strategies for real applications. On average, under the evaluated conditions, the lifetime of the system is double the time for a first non-fatal error to occur. The TMR architecture provides a warning feature, which is useful to estimate a failure rate and perform error recovery techniques. Furthermore, not all failures go through the Lockstep mode used by the architecture, around 1% of failures will not be warned beforehand. The cross-section of this type of errors are around 47 and 65 times lower than the cross-section of total errors, for experiments with protons of 15 MeV and 230 MeV respectively. Overall, the RISC-V fault tolerant proposal of AMD has a great reliability over faults in the configuration memory.

Funding

This work has been supported in part by the Community of Madrid, under grant PIPF-2024/TEC-34925, by project PID2022-138696OB-C21 funded by MCIN/AEI/10.13039/501,100,011,033 and by “ERDF A way of making Europe”, and in part by the European Union’s Horizon 2020 Research and Innovation Program through the Framework of the European Union (EU) Project RADiation facility Network for the Exploration of Effects for indusTry and research (RADNEXT) under Grant 101,008,126.

CRediT authorship contribution statement

Jorge Cano-Páez: Writing – original draft, Writing – review & editing, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Luis Entrena:** Writing – review & editing, Validation, Supervision, Resources, Funding acquisition, Project administration, Methodology. **Almudena Lindoso:** Writing – review & editing, Supervision, Resources, Funding acquisition, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

We would like to thank Centro Nacional de Aceleradores (CNA), Spain and Paul Scherrer Institute (PSI), Switzerland for providing the beamtime for the irradiation experiments.

Data availability

Data will be made available on request.

References

- [1] T. Oh, Y. Ha, D. Yoo, J. Yoo, AD-VILS: implementation and reliability validation of vehicle-in-the-loop simulation platform for evaluating autonomous driving systems, *IEEE Access* 12 (2024) 164190–164209.
- [2] G. Furano, S. Di Mascio, A. Menicucci, C. Monteleone, A European roadmap to leverage RISC-V in space applications, in: 2022 IEEE Aerospace Conf. (AERO), Big Sky, MT, USA, 2022, pp. 1–7.
- [3] Diligent, “CmodA7 Reference Manual Rev. B1”, 2019.
- [4] A. Aponte-Moreno, F. Restrepo-Calle, C. Pedraza, Reliability evaluation of RISC-V and ARM microprocessors through a new fault injection tool, in: 2021 IEEE 22nd Latin American Test Symp. (LATS), Punta del Este, Uruguay, 2021, pp. 1–6.
- [5] L. Sterpone, M. Violante, A new partial reconfiguration-based fault-injection system to evaluate SEU effects in SRAM-based FPGAs, *IEEE Trans. on Nuclear Sci.* 54 (4) (2007) 965–970.
- [6] P. Civera, L. Macchiarulo, M. Rebaudengo, M. Sonza Reorda, M. Violante, An FPGA-based approach for speeding-up fault injection campaigns on safety-critical circuits, *J. Electron. Test.* 18 (2002) 261–271.
- [7] L.A. Aranda, A. Sánchez-Macián, J.A. Maestro, ACME: a tool to improve configuration memory fault injection in SRAM-based FPGAs, *IEEE Access* 7 (2019) 128153–128161.
- [8] M. Alderighi, et al., Evaluation of single event upset mitigation schemes for SRAM based FPGAs using the FLIPPER fault injection Platform, in: 22nd IEEE International Symposium on Defect and Fault-Tolerance in VLSI Systems, 2007, pp. 105–113.
- [9] L. Entrena, M. García-Valderas, R. Fernandez-Cardenal, A. Lindoso, M. Portela, C. Lopez-Ongil, Soft error sensitivity evaluation of microprocessors by multilevel emulation-based fault injection, *IEEE Trans. on Comput.* 61 (3) (2012) 313–322.
- [10] AMD, “Soft error mitigation Controller product guide,” PG036, 2023.
- [11] C. Imianosky, D.A. Santos, D.R. Melo, F. Viel, L. Dillillo, Special session: reliability and performance evaluation of a RISC-V vector extension unit for vector multiplication, in: 2024 IEEE Int. Symp. on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT), Didcot, UK, 2024, pp. 1–6.
- [12] G. Cora, C. De Sio, D. Rizzieri, S. Azimi, L. Sterpone, A new reliability analysis of RISC-V soft processor for safety-critical systems, in: 2024 27th Int. Symp. on Design & Diagnostics of Electronic Circuits & Systems (DDECS), Kielce, Poland, 2024, pp. 31–36.
- [13] J. Laurent, V. Beroulle, C. Deleuze, F. Pebay-Peyroula, Fault injection on hidden registers in a RISC-V rocket processor and software countermeasures, in: 2019 Design, Automation & Test in Europe (DATE), Florence, Italy, 2019, pp. 252–255.
- [14] M.K. Minhas, H. Waris, S. Baloch, FVDCLS: functional verification of RISC-V based dual-core lockstep feature using fault injection mechanism, in: 2024 IFIP/IEEE 32nd Int. Conf. on Very Large Scale Integration (VLSI-SoC), Tanger, Morocco, 2024, pp. 1–4.
- [15] A.E. Wilson, M. Wirthlin, Fault injection of TMR open source RISC-V processors using dynamic partial reconfiguration on SRAM-based FPGAs, in: 2021 IEEE Space Computing Conf. (SCC), Laurel, MD, USA, 2021, pp. 1–8.
- [16] Á.B. de Oliveira, F.L. Kastensmidt, Evaluating fault-tolerant techniques on COTS RISC-V NOEL-V processor in Zynq UltraScale+ FPGA under proton testing, *IEEE Trans. on Nuclear Sci.* 70 (8) (2023) 1708–1715.
- [17] AMD, “MicroBlaze V Processor reference guide,” UG1629, 2024.
- [18] AMD, “MicroBlaze Processor Reference guide,” UG984, Nov. 2024.
- [19] AMD, “MicroBlaze Triple Modular Redundancy (TMR) Subsystem,” PG268, Apr. 2022.
- [20] R. Leveugle, A. Calvez, P. Maistri, and P. Vanhauwaert “Statistical fault injection: Quantified error and confidence,” Design, Automation & Test in Europe (DATE), Nice, France, 2009, pp. 502–506.



Jorge Cano-Páez received the B.S. degree in Computer Engineering from Universidad Complutense de Madrid (Spain) in 2023, and the M.S. degree in Electronics Systems Engineering from Universidad Carlos III de Madrid (Spain) in 2024. He is currently pursuing a PhD degree in Electric, Electronic and Automation Engineering from Universidad Carlos III de Madrid. His current research interests include fault tolerance, reconfigurable hardware and hardware acceleration.



Luis Entrena received the PhD degree from Universidad Politécnica de Madrid (Spain) in 1995. From 1990 to 1993 he was with AT&T Microelectronics at Bell Labs (USA). From 1993 to 1996 he was Project Leader at TGI (Spain). He is currently Full Professor at Universidad Carlos III de Madrid (Spain) where he has served as Head of the Electronic Technology Department and Director of the Postgraduate Program in Electrical Engineering. He has co-authored over 200 papers in journals and conferences, and two patents. His current research interests include online testing, fault tolerance, soft error mitigation, hardware security and hardware acceleration.



Almudena Lindoso (Senior Member, IEEE) received the M.S. degree in telecommunication engineering from the Universidad Politécnica de Madrid, Spain, in 2001, and the Ph.D. degree in electric, electronic and automation engineering from the University Carlos III of Madrid, Spain, in 2009. Since 2003, she has been a Professor and Researcher with the Electronic Technology Department, Universidad Carlos III de Madrid where she is currently an Associate Professor. She has co-authored over 80 publications in journals and conferences. Her research interests include hardware acceleration, image processing, and fault tolerance, and the adaptation in terms of reliability of high- performance commercial circuits for aerospace applications.